

Security Report — Login Bypass (TUDO)

1. What this report is about

I tested the TUDO web app (PHP + database). The goal was to show how an attacker can log in as another user without knowing their password. I used automatic scanning (Semgrep) and then checked the code by hand. At the end I ran my proof-of-concept script.

2. How I worked

1. I ran the app locally (Docker).
2. I ran Semgrep on the PHP code to find dangerous patterns.
3. I read the important files and connected Semgrep results to a real attack.
4. I showed two ways to bypass login (you can describe both if you did both).

3. What Semgrep found

Semgrep is a tool that searches code for common security problems. It reported that user input is mixed into an SQL string in `forgotusername.php`. That usually means SQL injection.

```
• → app git:(main) x semgrep --config auto .

Semgrep CLI

Scanning 26 files (only git-tracked) with:

✓ Semgrep OSS
  ✓ Basic security coverage for first-party code vulnerabilities.

✗ Semgrep Code (SAST)
  ✗ Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.

✗ Semgrep Supply Chain (SCA)
  ✗ Find and fix the reachable vulnerabilities in your OSS dependencies.

💡 Get started with all Semgrep products via `semgrep login`.
🔗 Learn more at https://sg.run/cloud.

100% 0:00:00

2 Code Findings

admin/import_user.php
>> php.lang.security.unserialize-use.unserialize-use
  (( Blocking ))
  Calling `unserialize()` with user input in the pattern can lead to arbitrary code execution.
  Consider using JSON or structured data approaches (e.g. Google Protocol Buffers).
  Details: https://sg.run/b24E

      7| $user = unserialize($userObj);

forgotusername.php
>>> php.lang.security.injection.tainted-sql-string.tainted-sql-string
  (( Blocking ))
  User data flows into this manually-constructed SQL string. User data can be safely inserted into SQL
  strings using prepared statements or an object-relational mapper (ORM). Manually-constructed SQL
  strings is a possible indicator of SQL injection, which could let an attacker steal or manipulate
  data from the database. Instead, use prepared statements (`$mysqli->prepare("INSERT INTO test(id,
  label) VALUES (?, ?)");`) or a safe library.
  Details: https://sg.run/LZYG

      12| $ret = pg_query($db, "select * from users where username='".$username."'");

Scan Summary

✓ Scan completed successfully.
• Findings: 2 (2 blocking)
• Rules run: 90
• Targets scanned: 26
• Parsed lines: ~100.0%
• Scan skipped:
  • Files matching .semgrepignore patterns: 114
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 90 rules on 26 files: 2 findings.
💡 Missed out on 1738 pro rules since you aren't logged in!
🔗 Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.

📦 A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading
• → app git:(main) x
```

4. Problem 1 — SQL injection (main path)

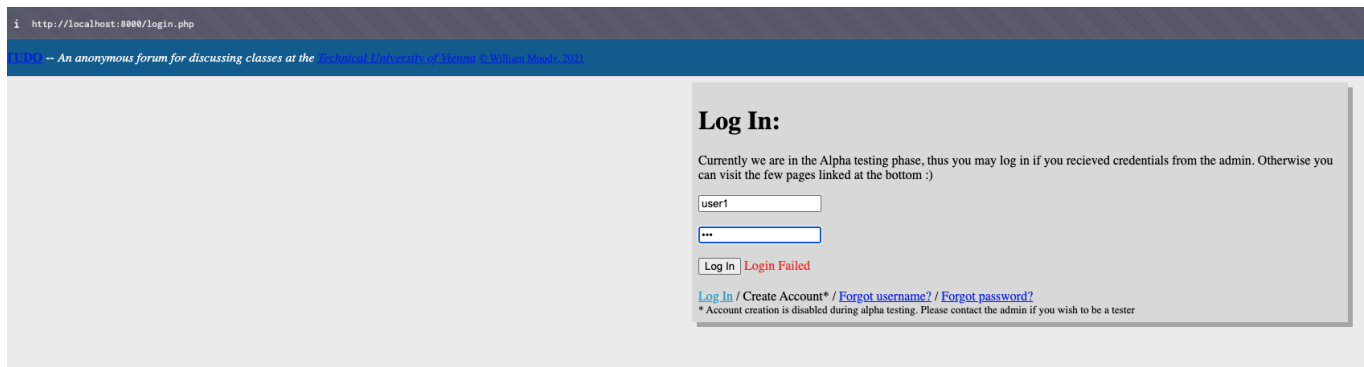
In forgotusername.php the username from the form is put inside the SQL query using string concatenation. An attacker can change the query.

```
forgotusername.php x
app > forgotusername.php
1  <?php
2      session_start();
3      if (isset($_SESSION['loggedin']) && $_SESSION['loggedin'] == true) {
4          header('location: /index.php');
5          die();
6      }
7
8      if ($_SERVER['REQUEST_METHOD'] === 'POST') {
9          $username = $_POST['username'];
10
11          include('includes/db_connect.php');
12          $ret = pg_query($db, "select * from users where username='".$username."'");
13
14          if (pg_num_rows($ret) === 1) {
15              $success = true;
16          } else {
17              $error = true;
18          }
19      }
20  ?>
21
```

The page answers “user exists” or “user doesn’t exist”. That answer works like a yes/no signal. Using many small questions, an attacker can slowly read secret data from the database.

5. How this leads to login bypass

When someone uses Forgot password, the server creates a reset token and saves it in the database. With SQL injection on Forgot username, an attacker can learn that token. Then they open Reset password, enter the token, and set a new password. After that they can log in as the victim.



```

→ lab_poc git:(main) x python3 blind_sqli_password_reset.py http://localhost:8000 user1 asd
[*] Requesting password reset for user1
[*] Brute-forcing uid via forgotusername.php oracle...
[*] uid=2
[*] Leaking token (32 chars, byte-by-byte)...
m60227Zqu_8A7J2tfGgPyaxEvyChLbVW
[*] Submitting new password to resetpassword.php...
[+] Done: log in as user1 with the new password.
→ lab_poc git:(main) x

```

1 http://localhost:8000/index.php

TUDO -- An anonymous forum for discussing classes at the Technical University of Darmstadt (TU Darmstadt)

[MoTD]

Hello, user1! Welcome to TUDO -admin :)





TU Freihaus TU Hauptgebäude TU Library

[All Posts]

Lva Code	Lva Name	Professor	ECTS	Comment
187.B12	Denkweisen der Informatik	Purgathofer, Peter	5.5	Very easy, but can be a bit frustrating
186.866	Algorithmen und Datenstrukturen	Kronegger, Martin	8.0	Pretty hard, but very interesting
184.735	Einführung in die Künstliche Intelligenz	Eiter, Thomas	3.0	Very lucky if you pass. Dont underestimate.
188.982	Privacy Enhancing Technologies	Weippl, Edgar	3.0	Very fun, and easy to get a perfect grade. Takes a lot of time.

[Create a post]

6. Extra detail — admin is protected here

The forgot-password page does not create a reset token for the user admin .

```

forgotpassword.php x
app > forgotpassword.php
1  <?php
2      session_start();
3      if (isset($_SESSION['loggedin']) && $_SESSION['loggedin'] == true) {
4          header('location: /index.php');
5          die();
6      }
7
8      if ($_SERVER['REQUEST_METHOD'] === 'POST') {
9          $username = $_POST['username'];
10
11         if ($username !== 'admin') {
12             include('includes/db_connect.php');
13             $ret = pg_prepare($db, "checkuser_query", "select * from users where username = '$username'");
14             $ret = pg_execute($db, "checkuser_query", array($_POST['username']));
15
16             if (pg_num_rows($ret) === 1) {
17                 $row = pg_fetch_row($ret)[0];
18
19                 include('includes/utils.php');
20                 $token = generateToken();

```

So this exact chain is for normal users, not for admin.

7. Problem 2 — weak random token (second path)

Reset tokens are built with `srand()` and the current time in milliseconds. That makes the token easy to guess if you try many seeds close to the moment of the reset.

```
utils.php x
app > includes > utils.php
1  <?php
2      function generateToken() {
3          srand(round(microtime(true) * 1000));
4          $chars = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_';
5          $ret = '';
6          for ($i = 0; $i < 32; $i++) {
7              $ret .= $chars[rand(0, strlen($chars)-1)];
8          }
9          return $ret;
10     }
11
12     class User {
13     public function __construct($u, $p, $d) {
14         $this->username = $u;
15         $this->password = $p;
16         $this->description = $d;
17     }
18     }
```

http://localhost:8000/login.php

[[[0]]] -- An anonymous forum for discussing classes at the [University of Illinois at Urbana-Champaign](#)

Log In:

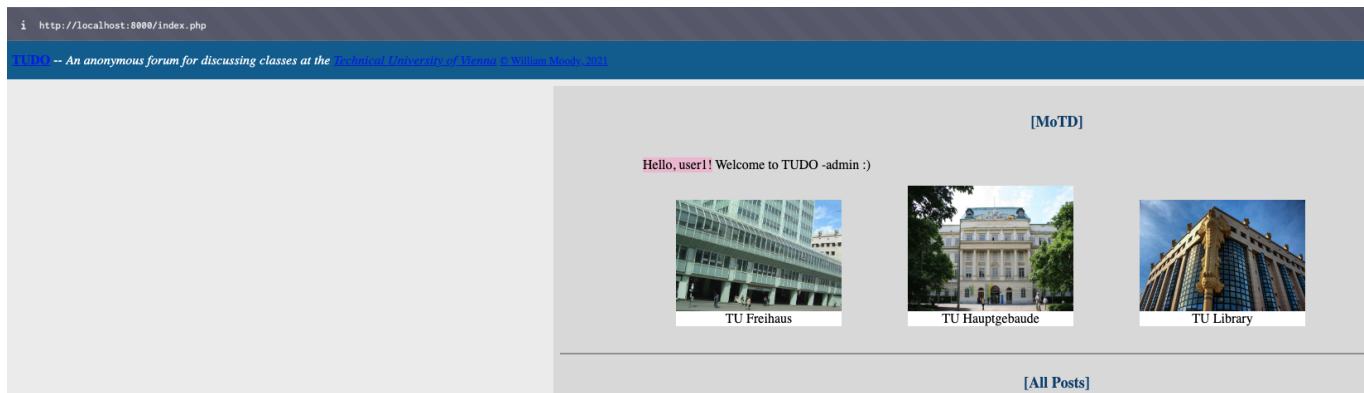
Currently we are in the Alpha testing phase, thus you may log in if you recieved credentials from the admin. Otherwise you can visit the few pages linked at the bottom :)

Login Failed

[Log In](#) / [Create Account*](#) / [Forgot username?](#) / [Forgot password?](#)

* Account creation is disabled during alpha testing. Please contact the admin if you wish to be a tester

```
lab_poc git:(main) x python3 guess_token_reset.py http://localhost:8000 user1 qwer
[*] Requesting password reset for user1
[*] Generating candidate seeds in [1777911830528, 1777911830584) via PHP (same as utils.php)...
[*] Candidates: 56
[+] Success - token matched weak RNG output.
[+] Login: user1 / qwer
lab_poc git:(main) x
```



8. How to fix it (short)

- Use prepared statements for all SQL (like in login.php), not string concatenation.
- Do not build secret tokens with srand / rand from the clock. Use a cryptographically secure random generator.
- Optionally show less difference in error messages to make blind attacks harder.

9. Conclusion

Semgrep pointed to dangerous SQL in forgotusername.php . Manual review showed how this connects to password reset and leads to authentication bypass. The weak random setup in utils.php gives a second bypass without SQL injection. Admin is partially protected from the reset flow, but normal users are still at risk.